

端口扫描

```
(root@kaada)-[/home/kali/Desktop]
└─# nmap -p- 192.168.56.141
Starting Nmap 7.99 ( https://nmap.org ) at 2026-06-28 09:07 +0000
Nmap scan report for 192.168.56.141
Host is up (0.00080s latency).
Not shown: 65532 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
2222/tcp  open  EtherNetIP-1
MAC Address: 08:00:27:CD:E4:E8 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 3.94 seconds

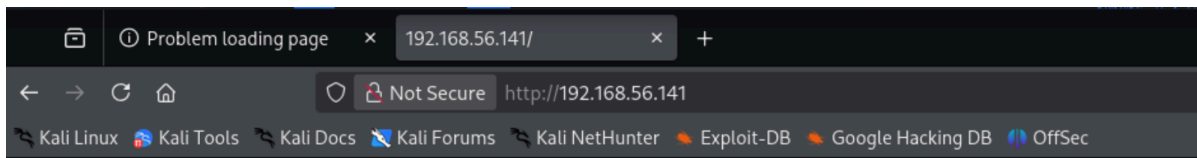

(root@kaada)-[/home/kali/Desktop]
└─# nmap -p22,80,2222 192.168.56.141 -sC -sV -T4 -A -O
Starting Nmap 7.99 ( https://nmap.org ) at 2026-06-28 09:10 +0000
Nmap scan report for 192.168.56.141
Host is up (0.00098s latency).

PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 10.0p2 Debian 7+deb13u4 (protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.67 ((Debian))
|_http-title: Site doesn't have a title (text/html; charset=UTF-8).
|_http-server-header: Apache/2.4.67 (Debian)
2222/tcp  open  ssh      OpenSSH 10.0p2 Debian 7+deb13u4 (protocol 2.0)
MAC Address: 08:00:27:CD:E4:E8 (Oracle VirtualBox virtual NIC)
warning: OSScan results may be unreliable because we could not find at least 1
open and 1 closed port
Aggressive OS guesses: Linux 4.15 - 5.19 (97%), Android 5 - 10 (Linux 3.4 - 3.18)
(93%), Linux 2.6.32 (93%), Google Chromecast or Roku TV (Linux 4.9) (93%),
Android 10 - 12 (Linux 4.14 - 4.19) (93%), Linux 5.10 - 5.19 (93%), Nintendo
Switch (93%), Linux 3.2 - 4.14 (93%), Linux 5.4 - 5.10 (93%), Openwrt 21.02
(Linux 5.4) (93%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 1 hop
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

TRACEROUTE
HOP RTT      ADDRESS
1   0.98 ms 192.168.56.141

OS and Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 10.64 seconds
```

访问80端口



Welcome

curl一下查看界面源代码

```
(root@kaada)-[/home/kali/Desktop]
└─# curl -i 192.168.56.141
HTTP/1.1 200 OK
Date: Sun, 28 Jun 2026 09:12:48 GMT
Server: Apache/2.4.67 (Debian)
Website: Maze-sec
Content-Length: 7
Content-Type: text/html; charset=UTF-8

welcome
```

发现自定义响应头Maze-sec

尝试带上该自定义响应头再次访问

```
(root@kaada)-[/home/kali/Desktop]
└─# curl -i -H 'website: Maze-sec' 192.168.56.141
HTTP/1.1 200 OK
Date: Sun, 28 Jun 2026 09:15:14 GMT
Server: Apache/2.4.67 (Debian)
Content-Length: 24
Content-Type: text/html; charset=UTF-8

challenge:challenge@2026
```

curl 是一个非常强大的命令行工具，用于通过网络与服务器传输数据。它支持几乎所有主流的网络协议（如 HTTP、HTTPS、FTP 等），是开发者进行 API 测试、数据抓取和文件下载的必备利器。

以下是 **curl** 最常用的核心命令参数：

- **-o / -O (Output):** 用于下载文件。小写 **-o** 需自定义保存的文件名，大写 **-O** 则直接保留远程原本的文件名。
- **-X (Request):** 指定 HTTP 请求的方法（如 **GET**, **POST**, **PUT**, **DELETE** 等）。若不指定，默认使用 **GET**。
- **-d (Data):** 用于发送 POST 请求的数据体，常用于提交表单或 JSON 数据。
- **-H (Header):** 添加自定义的 HTTP 请求头（例如指定数据格式：**-H "Content-Type: application/json"**）。
- **-I (Head):** 仅获取目标网址的 HTTP 响应头信息，不下载具体的网页正文。
- **-v (Verbose):** 开启详细模式，打印出完整的请求与响应过程，是排查网络问题的最佳帮手。
- **-L (Location):** 自动跟随服务器的网页重定向（例如遇到 HTTP 301/302 状态码时自动跳转）。

获得一组类似凭证的东西，尝试ssh登录到目标靶机

```
└─(root@kaada)-[/home/kali/Desktop]
└─# ssh challenge@192.168.56.141
challenge@192.168.56.141's password:
[sudo] password for challenge:
Sorry, try again.
[sudo] password for challenge:
bash-5.2# id
uid=0(root) gid=0(root) groups=0(root)
bash-5.2#
```

非常奇怪的是我们一登入就是root权限

```
bash-5.2# ls /
bin dev etc home lib lib64 proc tmp usr var
bash-5.2#
```

很明显这是一个类似容器的环境，但是没有docker，k8s的类似信息。

```

bash-5.2# ls /proc/
1      141 199 26 31 355 40 453 49 54 63 685      bus      dma
      irq      loadavg      partitions      sysvipc
10     142 2 267 32 356 41 46 493 55 631 687      cgroups
driver      kallsyms      locks      scsi      thread-self
107 15 20 27 33 359 42 461 494 56 642 688      cmdline
execdomains kcore      meminfo      self      timer_list
109 16 21 271 34 36 422 462 5 58 643 7      config.gz fb
      key-users      misc      slabinfo      tty
110 17 22 28 342 37 426 463 50 59 648 8      consoles
filesystems keys      modules      softirqs      uptime
111 18 23 281 345 377 43 464 502 6 662 acpi      cpuinfo fs
      kmsg      mounts      stat      version
12 183 231 285 346 38 436 465 51 60 665 asound      crypto
interrupts kpagecgroup mtrr      swaps      vmallocinfo
13 19 243 29 347 39 44 47 52 61 682 bootconfig devices
iomem      kpagecount net      sys      vmstat
14 194 25 3 35 4 45 48 53 62 683 buddyinfo diskstats
ioports      kpageflags pagetypeinfo sysrq-trigger zoneinfo

```

对比系统 PID 1 进程的根目录和本身的根目录

```

bash-5.2# ls -id / /proc/1/root
1835010 /      8807 /proc/1/root
bash-5.2#

```

可以确定当前是chroot

什么是 chroot? **chroot (change root)** 是一种 Linux/Unix 的系统机制，它能将一个进程及其子进程的“根目录”锁定在指定的子目录下，使其无法看到或访问系统里的其他文件。它常被用于构建最小化的测试环境、系统修复或基础的环境隔离。

[chroot jailbreak | 木头的小木屋](#)

[chroot&namespace逃逸总结 | C LBY's BLOG](#)

这里采用经典的proc逃逸法

```
bash-5.2# cd /proc/1/root
bash-5.2# ls
bin  dev  home          initrd.img.old  lib64          media  opt   root  sbin  sys
usr  vmlinuz
boot  etc  initrd.img  lib              lost+found  mnt    proc  run   srv   tmp
var  vmlinuz.old
bash-5.2# cd root
bash-5.2# ls
give_flag  give_pass  pass.txt  root.txt
bash-5.2# cat pass.txt
HHLxQsx2qnbuDL7js*c5
bash-5.2# cat root.txt
flag{root-9ff99c5832fe3e5bb9f6bd363c186612}
bash-5.2#
```